

How to NOT depend on 3rd parties when consuming APIs (and make your life easier when producing)



Tobias Fenster

directions poznan
emea 2025

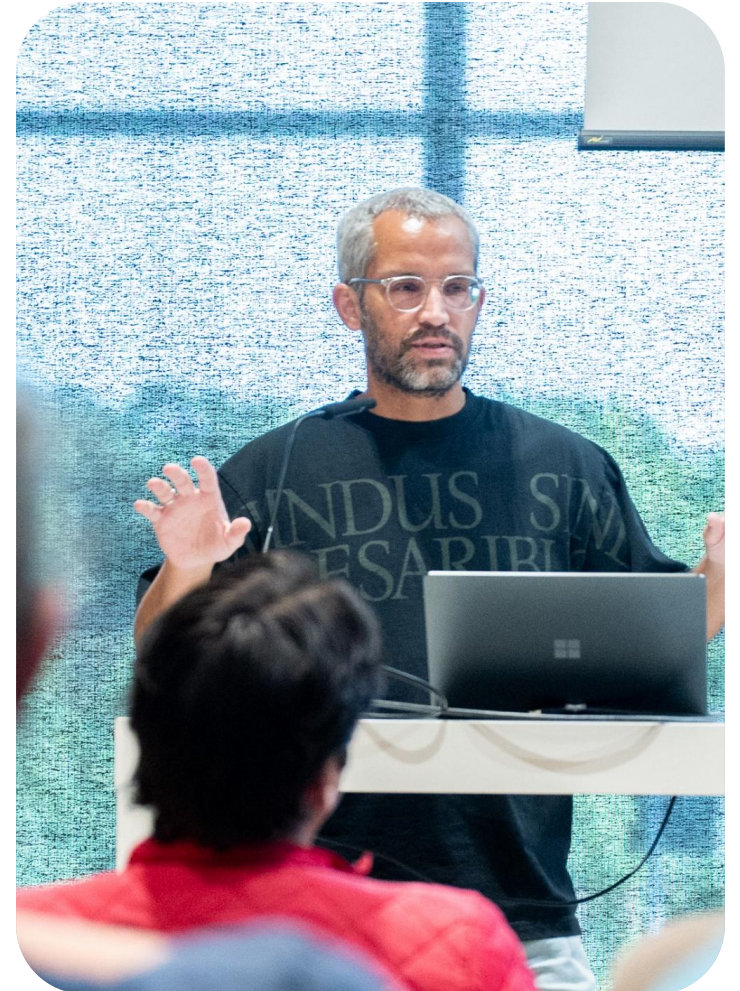
Business

General Manager at 4PS Germany
Chief Engineer at HILTI

Community

Microsoft MVP for Business Central and Azure; Regional Director
Docker Captain
Blog and podcast "Window on Technology" at tobiasfenster.io

Find me and let's connect on LinkedIn and Bluesky



Agenda

Automatic generation of REST API clients

Mocking Business Central & 3rd party backends

Q&A

Agenda

Automatic generation of REST API clients

Mocking Business Central & 3rd party backends

Q&A

Disclaimer

directions poznan
emea 2025



A lot of the content and especially the code around Kiota is created by my colleague

Simon Fischer

Find him at simonOfHH.tech or on LinkedIn and Bluesky and talk to him if you like it!

The challenge: API No. 3928

Now (fortunately) many integrations are API-based

Handling in AL improved

Nevertheless: a lot of repetitive code is necessary that does not carry any business logic

Changing, expanded APIs require manual maintenance

The challenge: API with BC are not fun

Typically not very dev-friendly (error-prone work with JSON objects or temporary tables)

```
JToken.ReadFrom(Data);
JObject := JToken.AsObject();
JObject.SelectToken('results', JToken);
JArray := JToken.AsArray();

foreach JToken in JArray do begin
    NYTBestSellersTheme.Init();
    NYTBestSellersTheme."List Name" := CopyStr(GetValueAsText(JToken, 'list_name'), 1, MaxStrLen);
    NYTBestSellersTheme."List Name Encoded" := CopyStr(GetValueAsText(JToken, 'list_name_encoded'), 1, MaxStrLen);
    NYTBestSellersTheme.Updated := CopyStr(GetValueAsText(JToken, 'updated'), 1, MaxStrLen(NYTBestSellersTheme.Updated));
    NYTBestSellersTheme."Newest Published Date" := DT2Date(EvaluateUTCDateTime(GetValueAsText(JToken, 'newest_published_date')));
    NYTBestSellersTheme."Oldest Published Date" := DT2Date(EvaluateUTCDateTime(GetValueAsText(JToken, 'oldest_published_date')));
    NYTBestSellersTheme.Insert();
end;
end;
```

```
JsonOrder.Get('Lines', JsonTokenLines);
JsonArrayLines := JsonTokenLines.AsArray();

foreach JsonTokenLine in JsonArrayLines do begin
    | Message(format(JsonTokenLine));
end;
```

The challenge: API with BC are not fun

A lot of repetitive work to map data structures

```
// To store the content in JSON Object
JObject.ReadFrom(Result);
JObject.Get('name', Jtoken);
API.Name := Jtoken.AsValue().AsText();
JObject.Get('email', Jtoken);
API.Email := Jtoken.AsValue().AsText();
JObject.Get('username', Jtoken);
API."User Name" := Jtoken.AsValue().AsText();
JObject.Get('street', Jtoken);
API.Street := Jtoken.AsValue().AsText();
```


The challenge: API with BC are not fun

Certain steps (authentication, HTTP header handling, content structure, etc.) still partially reimplemented each time (improved since AJ Kauffmann's Rest Client Module).

```
HttpRequestMessage.Method := 'POST';  
HttpRequestMessage.SetRequestUri('ptsv2.com/.../post');  
  
HttpContent.GetHeaders(HttpHeadersContent);  
HttpHeadersContent.Clear();  
HttpHeadersContent.Add('Content-Type', 'application/json');  
  
HttpRequestMessage.GetHeaders(HttpHeadersRequestMessage);  
HttpHeadersRequestMessage.Add('Cache-Control', 'no-cache');  
HttpHeadersRequestMessage.Add('Authorization', 'bearer ' + Token);  
  
JsonObject.Add('Kind', 1);  
JsonObject.Add('ActorCSID', 74);  
JsonObject.Add('ProdConceptID', 38);
```

But APIs with BC could be fun!

Instead of

```
procedure ReadItemNumberSchemaIdentifierFromProjectJson(jsonBody: JsonToken): Text
var
    subTokenProject, subTokenProjectInfo, subTokenItemNumberSchema, subTokenIdentifier : JsonToken;
begin
    if jsonBody.SelectToken('project', subTokenProject) then
        if subTokenProject.SelectToken('projectInformation', subTokenProjectInfo) then
            if subTokenProjectInfo.SelectToken('itemNumberSchema', subTokenItemNumberSchema) then
                if subTokenItemNumberSchema.SelectToken('identifier', subTokenIdentifier) then
                    exit(subTokenIdentifier.AsValue().AsText());
                exit('');
            end;
```

we could have

```
procedure ReadItemNumberSchemaIdentifierFromProject(Project: Codeunit "ProjectDto"): Text
begin
    exit(Project.ProjectInformation().ItemNumberSchema().Identifier());
end;
```

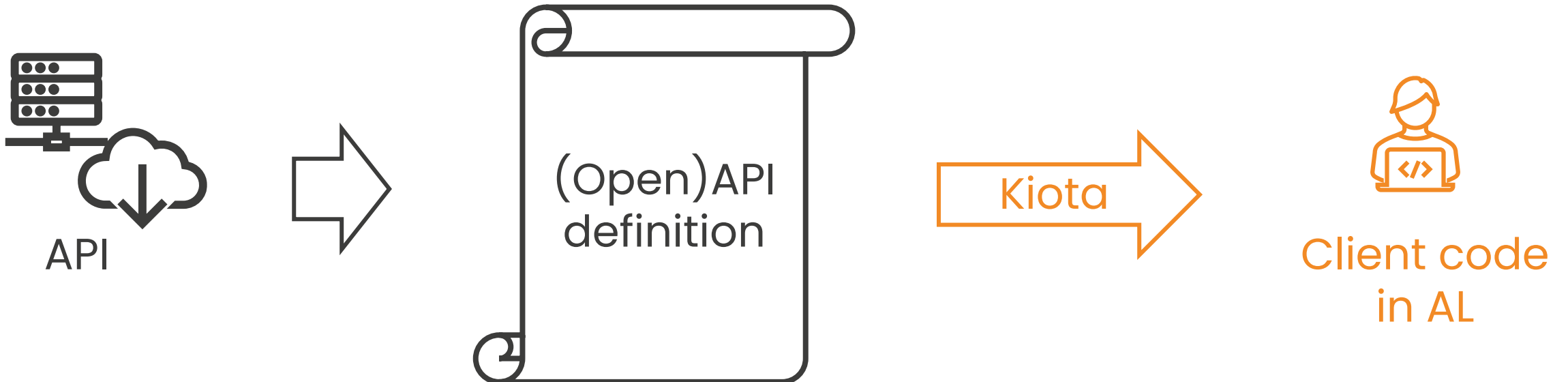
One solution: Kiota generates clients

Instead of manually written and maintained code: generate clients with Kiota

Idea: Use an OpenAPI definition of an API to generate a client

Not necessarily entire API, can also be only part of the definition

Generation in different programming languages possible



Demo

Create an AL API client with Kiota

Demo API is „PetStore“, the (sometimes shaky) default sample for OpenAPI:

- CRUD for pets
- Also things like tags and status for pets or ordering pets, but we won't use that today

Demo: What happened?

Extension with client and helper objects was generated, including a dependency to an abstractions library (generic request handling, authorization, interfaces, helper classes)



API Client: basic functionality and high-level entity access



Request Client classes: handle actual requests



Model classed per entity: Wrappers for easy data access

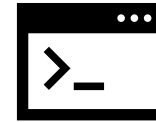
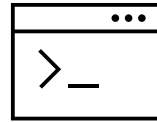
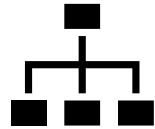


Depend on Kiota.Abstractions AL app

Demo: What happened behind the scenes?

directions
emea 2025

Kiota flow:



OpenAPI

- v2/v3
- YAML/JSON

URL Space

- Resources
- Relations
- Operations

Generic Code Model

- Classes
- Properties
- Methods

Language-specific
refinement

Source Code

- C#
- Java
- TypeScript
- AL
- ...

Where we are and what we need

Generation code is there, but not „production ready“ yet → Would love your test and input

<https://github.com/SimonOfHH/kiota>

Companion app is there, but should be in Microsoft's system app → Need your feedback and support

<https://github.com/SimonOfHH/Kiota.Abstractions-al>

We have a sample and some have used it, but the more the better → Give it a try, let us know where you have issues and share

<https://github.com/SimonOfHH/kiota-al-samples>

Would be great to get the overall tooling „production ready“ (e.g. support for continuous client updates through pipelines) and AL support for the VS Code extension as well → Help if you can

Automatic generation of REST API clients

Mocking Business Central & 3rd party backends

Q&A

Agenda

The challenge: Dependency on 3rd parties

We need to integrate more and more systems (you've heard that before...) via APIs

We also integrate BC into more and more other systems

During development and automated tests, we often face challenges

- Availability might not be easy
- Data often is an issue
- Scaling isn't easy: What if 3 pipelines run at the same time?

"Mocking" can be a solution: Instead of the real BC or 3rd party system, you have a "mock", which is a system that behaves in the same way

- For an API, that means it accepts the same (relevant) requests and send the same responses

What is **WIREMOCK**?

<https://github.com/wiremock/wiremock>:

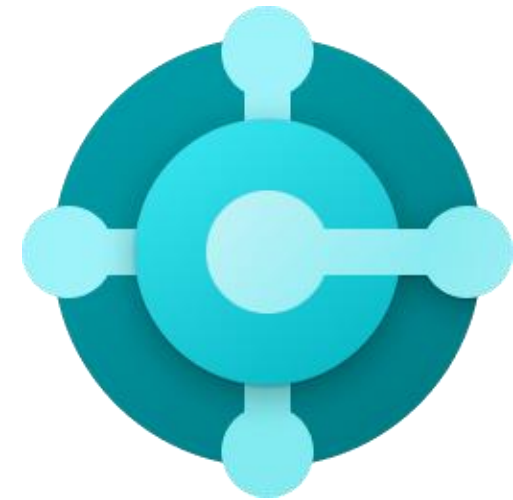
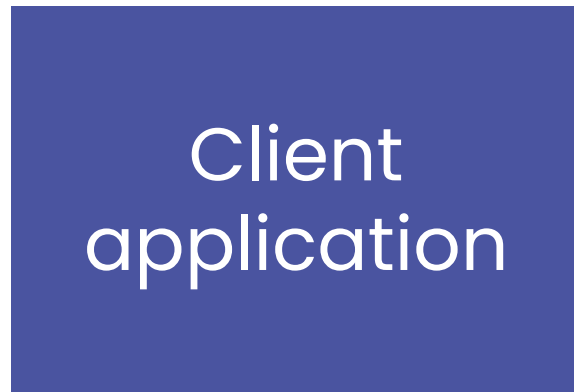
WireMock is a popular open-source tool for API mock testing with over 5 million downloads per month. It can help you to create stable test and development environments, isolate yourself from flaky 3rd parties and simulate APIs that don't exist yet.

Key features:

- Mock APIs by defining them via JSON or code
- Record API calls and create mocks from that
- Stateful behavior simulation

WireMock for BC mocking

Scenario 1: Use WireMock to mock BC



WireMock for BC mocking

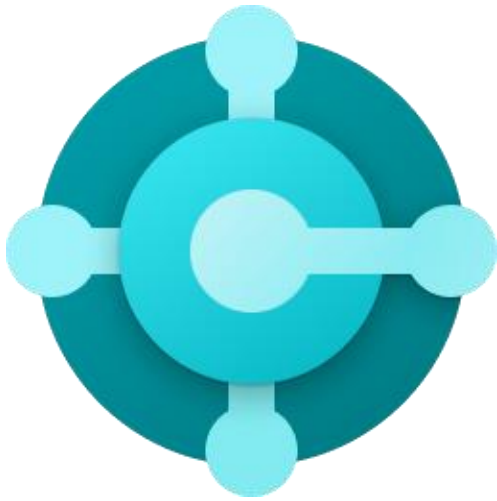
Scenario 1: Use WireMock to mock BC



Example: Get companies, create and read customer

WireMock for 3rd party mocking

Scenario 2: Use WireMock to mock 3rd party applications



3rd party
application

WireMock for 3rd party mocking

Scenario 2: Use WireMock to mock 3rd party applications



Example: CRUD actions for the PetStore application

Demo: Mock BC

Mock responses to:

- GET /companies
- GET /companies(<id>)/customers
- GET /companies(<id>)/customers(<id>)
- POST /companies(<id>)/customers

DEMO!

Done with “standard” WireMock in Java using recorded and optimized JSON files

Client is tiny C# Console application

- Dev and test
- Dev done in devcontainers: 1 for client application, 1 for WireMock → Reach out to me if you want to learn more, today we won't have enough time

Demo: What happened?

Set up WireMock for recording

Record the required steps

Optimize: Rename, dynamic values

Stateful scenario with create, update and delete

Usage of the mock in development and test development

Closing remarks

We have only scratched the surface for WireMock. E.g. there is a .NET variant, it can also do a bit of “chaos engineering” to induce faults and delays and the coding and recording options have a lot of benefits we haven’t seen today

There are of course alternatives like the recently introduced feature in AL

- If you only do AL → WireMock is probably not the best fit
- If you are not afraid of a little JSON / Java / C# and/or you have additional clients with a need to mock BC → WireMock is worth a look!

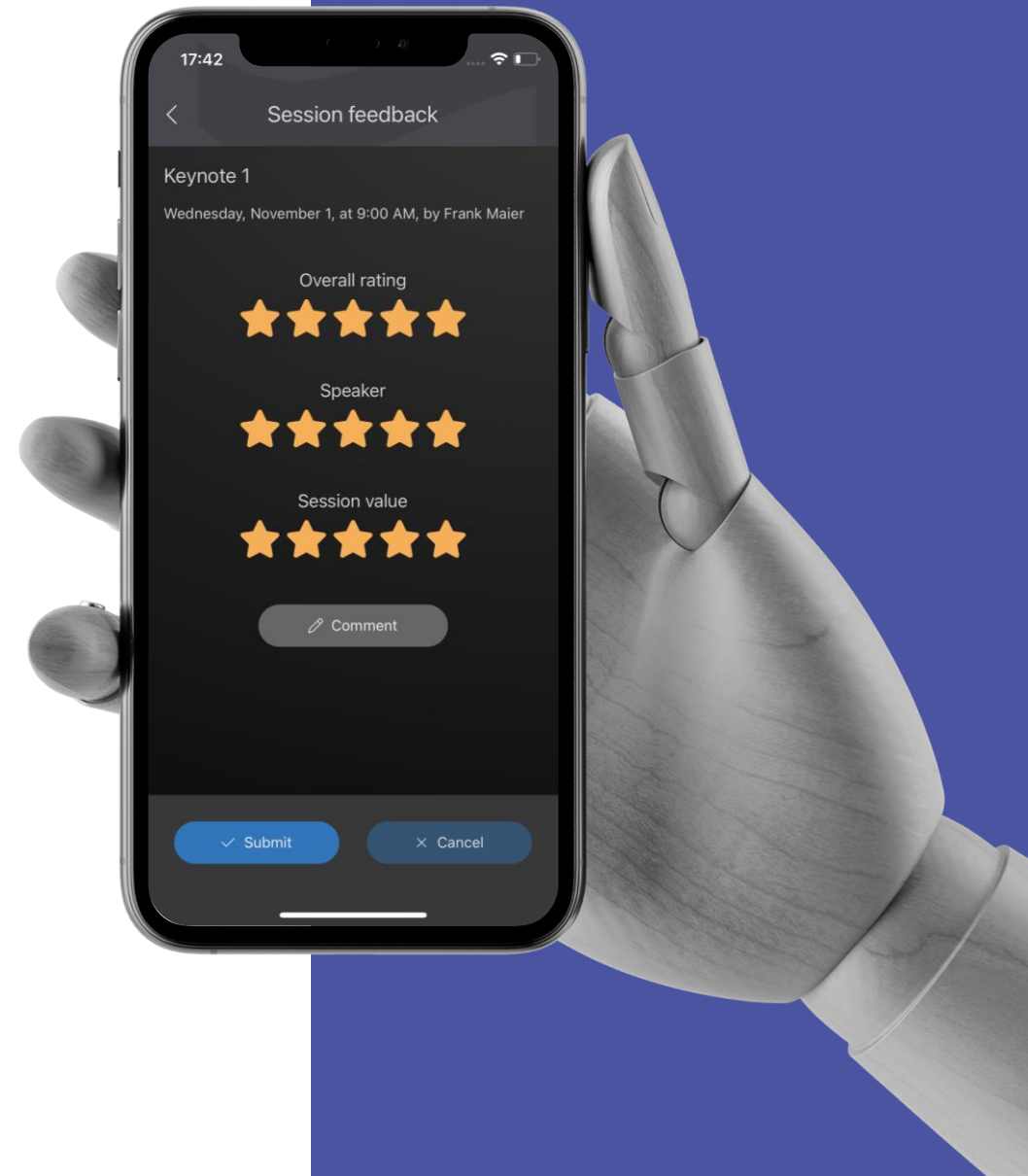
Give us Feedback!

and help us improve!

Please rate my session in the Conference App!

and leave a constructive comment.

Thank you !



Any questions I can try to answer?

